Journaling File Systems

Crash Consistency, Recovery and the ext4 Case Study

Prashant Jadav

DEPSTAR – CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

1. Explain the need for journaling in file systems.
2. Describe the structure and operation of a journal.
3. Differentiate metadata journaling and data journaling.
4. Explain writeback, ordered and journal data modes.
5. Trace crash recovery using transaction commit records.
6. Analyze journaling in Linux ext4 using a practical case study.
7. Evaluate the performance and consistency trade-offs of journaling.

Prerequisite concepts: Inodes, directories, file metadata, disk blocks and file-system allocation.

Why Is File-System Consistency Important?

A file-system operation often modifies multiple on-disk structures.

Example: Creating a new file

1. Allocate an inode for the file.
2. Allocate a data block when required.
3. Update the inode bitmap.
4. Update the inode metadata.
5. Add a directory entry pointing to the inode.

These changes are not necessarily written to disk simultaneously.

The Problem

If power fails after some updates reach disk but before the others, the file system may contain inconsistent metadata.

Why Is File-System Consistency Important?

Possible consequences

- A directory entry refers to an invalid inode.
- An allocated block is not referenced by any file.
- A file references a block marked as free.
- File contents or metadata become inconsistent.

The Crash-Consistency Problem

Suppose a file creation operation requires three logical updates:

$$T = \{D, I, B\}$$

where:

- D: Insert a directory entry.
- I: Initialize the inode.
- B: Mark the inode as allocated in the bitmap.

Assume the intended final state is:

$$D = 1, \quad I = 1, \quad B = 1$$

Here, 1 means that the corresponding structure has been updated correctly.

The Crash-Consistency Problem

State	D	I	B	Interpretation
Initial	0	0	0	File absent
Valid	1	1	1	File created
Crash A	1	0	0	Directory points to invalid inode
Crash B	0	1	1	Allocated inode may be orphaned

Key insight: Correct individual writes do not automatically guarantee a consistent multi-structure operation.

What Is a Journaling File System?

Definition

A journaling file system records intended file-system changes in a special area called a **journal** or **log** before applying or committing the corresponding updates to their normal locations.

The journal enables the FS to determine which operations were committed when a crash occurred.

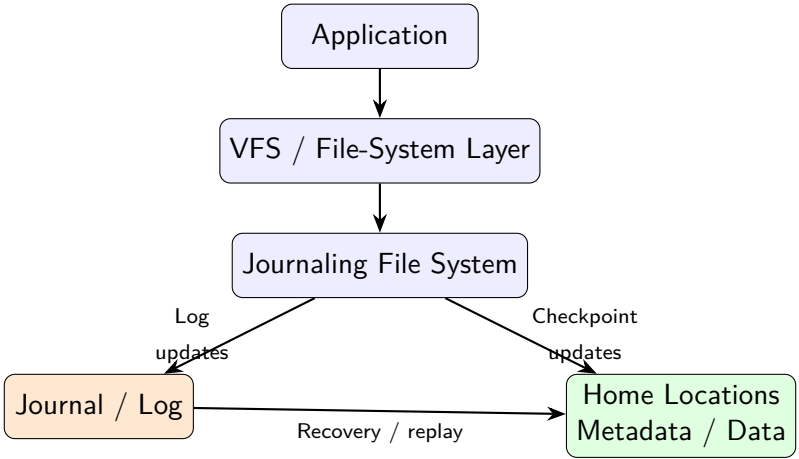
Basic principles

- Group related updates into a transaction.
- Record the required information in the journal.
- Mark the transaction as committed.
- Apply or checkpoint the updates to their home locations.
- Replay committed transactions during recovery when necessary.

Important distinction

Journaling improves crash consistency and recovery. It does not, by itself, guarantee that every recent user write survives a crash.

Journal Architecture



Journal Architecture

- **Journal:** Holds records describing transactions.
- **Home locations:** Normal disk locations of inodes, bitmaps, directories and possibly file data.
- **Checkpointing:** Writes committed changes to their normal locations so journal space can eventually be reused.

How Does a Journal Transaction Work?

A simplified metadata-journaling transaction follows these steps:

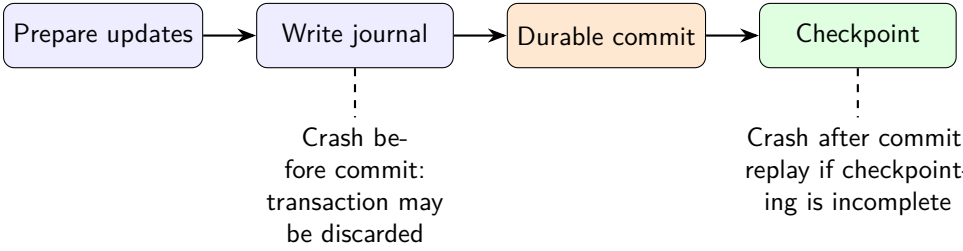
1. **Begin:** Identify the file-system operation.
2. **Record updates:** Store the relevant metadata changes or their journaled representations.
3. **Commit:** Persist a valid commit record after the required transaction records.
4. **Checkpoint:** Write the committed changes to their normal disk locations.
5. **Reclaim:** Reuse journal space when the transaction is no longer needed for recovery.

Commit point

A transaction is considered committed only when its commit record has been made durable according to the file system's ordering and storage guarantees.

Implementation note: Real systems may use checksums, sequence numbers, descriptor records, revocation records and multiple journal transactions. The sequence above is conceptual.

Write Protocol: Before and After a Crash



Core recovery rule

- A transaction without a valid durable commit record must not be treated as a successfully committed transaction.
- A committed transaction may need to be replayed if its changes have not all reached their home locations.

What Happens When the System Crashes?

Consider a transaction that updates a directory entry and an inode.

Crash before commit

1. Journal records may be incomplete.
2. The transaction has no valid commit record.
3. Recovery does not replay it as committed.
4. FS checking may still be needed for unrelated or pre-existing inconsistencies.

Crash after commit

1. A valid commit record exists.
2. Some home-location writes may be incomplete.
3. Recovery identifies the committed transaction.
4. Required updates are replayed or otherwise recovered.

Caution

This is a simplified model. Actual recovery behavior depends on the file system, journaling mode, transaction format and storage guarantees.

Three Common Journaling Modes

The principal modes differ in what information is journaled and when file data is written.

Mode	Journal contents	Data ordering	Typical trade-off
Writeback	Metadata	No guarantee that new file data reaches disk before metadata	Higher performance potential; weaker ordering
Ordered	Metadata	Relevant data is written before associated metadata is committed	Balance of performance and consistency
Journal	Metadata and file data	Data and metadata are journaled before checkpointing	Stronger transactional data protection; more I/O

- All three modes use journaling, but they do not provide identical guarantees for file contents.
- In ext4, these modes are commonly specified using the `data=` mount option.

Mode 1: Writeback Journaling

What is journaled? Primarily file-system metadata.

Simplified sequence

1. Metadata changes are recorded in the journal.
2. The metadata transaction is committed.
3. Metadata is written to its normal location.
4. File data blocks may be written independently.

Crash scenario

A file is extended, and its new metadata reaches disk before the new data block is fully written. Following a crash, the file may expose stale or incomplete contents in the newly visible region.

Advantages

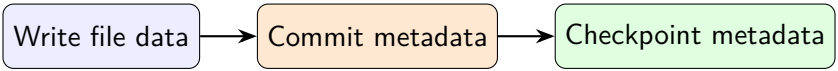
- Avoids the cost of writing file data twice to the journal and home location.
- Can offer better write performance.

Limitation: Metadata recovery does not imply that the latest file data is intact or that writes are correctly ordered.

Mode 2: Ordered Journaling

What is journaled? Primarily metadata, with an ordering constraint on relevant file data.
Simplified sequence

1. Prepare the new file data blocks.
2. Ensure relevant data blocks are written before the associated metadata transaction is committed.
3. Commit the metadata transaction to the journal.
4. Checkpoint metadata to its home locations.



Why is ordering useful?

It reduces the risk that newly committed metadata exposes file data that was not written first.

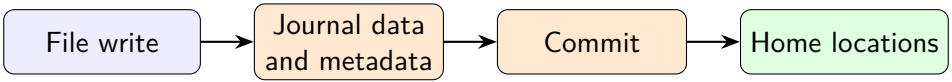
Limitation: It does not journal every file-data write and does not guarantee that all recently written user data survives a crash.

Mode 3: Full Data Journaling

In full data journaling, both file data and metadata are recorded in the journal.

Simplified sequence

1. Record the relevant file data in the journal.
2. Record the corresponding metadata changes.
3. Commit the transaction.
4. Copy the committed data and metadata to their home locations.



Advantages

- Provides a stronger transaction-based recovery mechanism for journaled file data.
- Can protect against partial application of journaled writes.

Costs

- More write traffic and journal bandwidth consumption.
- Potentially lower throughput for data-intensive workloads.

Crash Recovery Algorithm

The recovery process inspects the journal to identify valid committed transactions.

Conceptual algorithm

1. Locate and validate the journal.
2. Scan journal records and identify transaction boundaries.
3. Validate transaction records and commit status.
4. Select committed transactions requiring recovery.
5. Replay the required updates in the prescribed order.
6. Complete recovery and allow normal file-system operation.

Crash Recovery Algorithm

Pseudocode

```
Recover(Journal):
    records = scan_and_validate(Journal)

    for each transaction T in log order:
        if T has valid durable commit record:
            if T requires replay:
                replay(T)
        else:
            do not replay T as committed

    complete_recovery()
```

This is an explanatory algorithm, not source code for a particular file system. Real implementations handle checksums, transaction identifiers, wraparound and checkpoint state.

Case Study: Linux ext4

ext4 is a widely used Linux file system that supports metadata journaling through the JBD2 journaling layer.

Scenario

A Linux workstation is creating a report named `report.txt`. The operation involves directory metadata, inode information and possibly new data blocks.

Relevant ext4 concepts

- **Inode:** Stores file metadata and references to its data blocks.
- **Directory entry:** Maps a file name to an inode number.
- **Journal:** Records transactional file-system updates.
- **JBD2:** The journaling layer used by ext4.
- **Checkpointing:** Transfers committed journaled updates to their normal locations.

Important detail: ext4 commonly uses ordered data mode by default, although mount configuration and system setup can change it. The precise behavior also depends on the operation and kernel version.

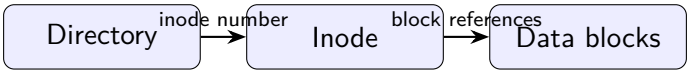
ext4 Case Study: Creating a File

Suppose the user executes:

```
echo "Operating Systems" > report.txt
```

A simplified view of the required updates is:

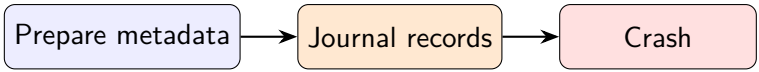
1. Resolve the destination directory.
2. Allocate or initialize an inode for `report.txt`.
3. Update the directory entry.
4. Allocate data blocks if needed and write file contents.
5. Journal the relevant metadata updates.
6. Commit the transaction and checkpoint metadata.



The journal helps maintain consistency among related metadata changes. It does not mean that every byte written by the application is always stored in the journal.

ext4 Case Study: Crash Before Commit

Assume the system crashes after journal records have been prepared but before a valid durable commit record is written.

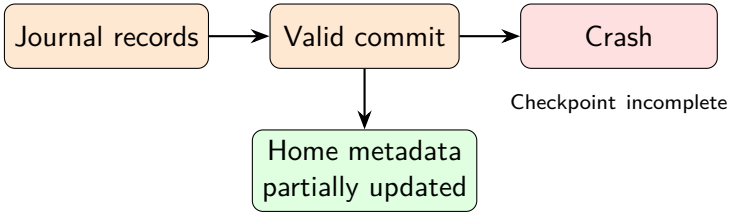


Expected recovery behavior

- The incomplete transaction is not replayed as committed.
- The file-system recovery mechanism restores the metadata state according to the journal's recovery rules.
- The file creation may not survive as a completed operation.

ext4 Case Study: Crash After Commit

Now suppose the metadata transaction has a valid durable commit record, but the system crashes before all metadata reaches its home locations.



Expected recovery behavior

1. The journal is examined during recovery.
2. The committed transaction is recognized.
3. Updates that require replay are applied.
4. The resulting metadata is made consistent according to the journal protocol.

Conclusion: The journal provides a recovery record that helps complete a committed operation even when checkpointing was interrupted.

Worked Example: Three Metadata Updates

Assume a transaction modifies:

$$T = \{D, I, B\}$$

where D is a directory entry, I is inode metadata, and B is an allocation bitmap.
Initially:

$$(D, I, B) = (0, 0, 0)$$

After a valid transaction commit, the intended state is:

$$(D, I, B) = (1, 1, 1)$$

Worked Example: Three Metadata Updates

Suppose the crash occurs after only the directory entry reaches its home location.

Stage	D	I	B
Before operation	0	0	0
Journal transaction committed	0	0	0
Partial checkpoint	1	0	0
After recovery	1	1	1

Interpretation

The table illustrates the intended result when all three updates belong to one correctly journaled transaction. Actual on-disk states and recovery details depend on the implementation.

Journaling vs. fsck-Based Recovery

Traditional non-journaling file systems may require a consistency checker to scan structures after an unclean shutdown.

Aspect	**Journaling**	**Traditional checking**
Primary approach	Recover from transaction records	Inspect file-system structures
Recovery work	Often proportional to relevant journal work	Can require extensive structure scanning
Consistency mechanism	Commit records and replay rules	Detection and repair of inconsistencies
Limitation	Does not guarantee all user data survives	Repairs may be time-consuming and cannot reconstruct lost data

Clarification: Journaling does not eliminate every need for file-system checking. Tools such as `e2fsck` remain relevant for corruption, unsupported conditions and certain repair scenarios.

Advantages and Limitations

Advantages

- Faster recovery after many crashes.
- Improved metadata consistency.
- Reduced need for full-volume checking after routine crashes.
- Better handling of partially applied metadata updates.
- Predictable transaction-based recovery.

Limitations

- Additional write op. and I/O overhead.
- Journal space and bandwidth requirements.
- Possible performance bottlenecks.
- Does not prevent physical disk failure.
- Does not replace backups.
- Does not guarantee application-level atomicity.

Storage durability

Correct recovery depends on the required writes and commit records actually reaching durable storage in the prescribed order. Hardware caches, flush operations and device behavior matter.

Performance Considerations

Journaling affects performance in several ways.

1. Write amplification

In full data journaling, file data may be written once to the journal and again to its home location.

$$\text{Additional I/O} \approx \text{Journal writes} + \text{Checkpoint writes}$$

Aactual overhead depends on caching, batching, workload and implementation.

2. Commit latency

A transaction may need to wait for its commit record to become durable.

3. Group commit

Several transactions may share a commit operation or durability boundary, improving throughput under suitable workloads.

4. Checkpointing

Large write workloads can generate more journal traffic than the storage device can efficiently absorb.

Practical Linux Demonstration

Use a disposable virtual machine or a non-critical test file system.

Step 1: Identify the file system

```
findmnt -no SOURCE,FSTYPE,OPTIONS /
```

Step 2: Inspect ext4-related mount options

```
findmnt -t ext4 -o TARGET,SOURCE,FSTYPE,OPTIONS
```

Look for an option such as `data=ordered`, if reported.

Step 3: Create and inspect a test file

```
mkdir -p /tmp/journal-demo
echo "FOSD journaling experiment" \
    > /tmp/journal-demo/report.txt
cat /tmp/journal-demo/report.txt
stat /tmp/journal-demo/report.txt
```

Step 4: Discuss crash consistency

Identify which operations involve file data and which involve metadata.

Common Misconceptions

1. **“Journaling prevents all data loss.”**
   False. Recent user data may still be lost, especially when it has not been committed or made durable.
2. **“All journaling modes journal file contents.”**
   False. Metadata-only modes journal metadata; full data journaling also records file data.
3. **“A successful write system call always means data is on physical storage.”**
   False. Data may remain in caches unless the required durability operation has completed.
4. **“A committed transaction must already be present at every home location.”**
   False. It may require replay after a crash during checkpointing.
5. **“Journaling replaces backups.”**
   False. Journaling addresses consistency and recovery, not recovery from deletion, ransomware or complete device failure.

Summary

- File-system operations often modify multiple related metadata structures.
- A crash during these updates can leave inconsistent state.
- Journaling records transactions and uses commit information to support recovery.
- Committed transactions may be replayed after a crash.
- Writeback, ordered and full data journaling provide different ordering guarantees and incur different costs.
- Linux ext4 uses JBD2 and commonly operates in ordered data mode unless configured otherwise.
- Journaling improves consistency, but is not a substitute for backups, application-level transactions or durable-write protocols.

Review Questions

1. Why can creating a file require several metadata updates?
2. What is the role of a journal commit record?
3. Differentiate ordered journaling and full data journaling.
4. What happens if a crash occurs after commit but before checkpointing?
5. Why does journaling not guarantee that recent application data survives a power failure?
6. Explain the role of JBD2 in Linux ext4.
7. Which journaling mode would you choose for a workload prioritizing throughput, and what trade-off would you accept?

Case-study question

A transaction has committed three metadata updates, but only one has reached its home location when the system crashes. Explain how journal recovery can restore the intended consistent state.